

the initialisation of the respective controller. Figure 7 shows the main set-up menu for the RAID controller in Slot 11. Pay attention to the last option, *Select as Boot Controller*. This option is responsible for setting the boot priority among RAID disks. If you selected this option, then the loader will begin using this particular disk for its purposes. Therefore, even if you've correctly selected all of the previous options in the ROM-based Setup Utility, and forgotten about this tricky option, then expect your system to not be bootable. But don't panic. We know how to restore the MBR, and we'll do just that. Let's save private GRUB!

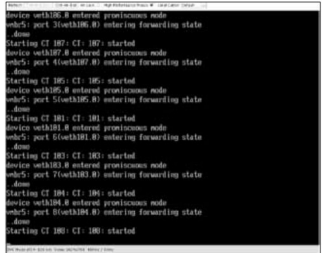
Get my data back!

To rescue all user data as well as all the Linux systems configuration, we could grab ourselves any Live CD, boot off it, and copy the files to SAN storage. After that, we could re-install the entire system from scratch. However, I think this approach is wrong. Instead of copying several thousand gigabytes from RAID arrays via the local network, we can save our time. The first thing we can avoid is the need to integrate network drivers into the Live CD. This particular HP model has 2 Gigabit NICs, and once these drivers are put into the *initramfs* file, all we need is to find and use them. Otherwise, you'll have to change the network topology—find a person who will switch cables from one Ethernet port to another. That's not easy to explain to a stranger who is miles away from you, in the middle of the night, especially when the server room hosts dozens of communication racks and cabinets.

Let's make sure that the drives on the first RAID controller (where we store the data and the system) are consistent. In order to do this, continuously press the F8 button right after the controller initialisation of Slot 0 to get into the RAID configuration. According to the layout visible in Figure 8, the data we need has remained untouched on the disks. This is great; full speed ahead! Our task is to boot the system, then transfer the */boot/* contents onto our PC. Only after that will we create a bootable ISO image with the kernel and *initramfs* files rescued from the system. This ISO image will load the whole system as it was before the crash. Once the system is loaded fully, we will try to re-run the last command, *grub-install*. Easier said than done, right? Let's see.

I used the kernel option *init=/bin/sh*, when I started a usual installation of Debian Lenny. This allows me to have a console in run-level 1, after all kernel initialisation and system configuration takes place. Let's see exactly what virtual devices are connected to this HP server:

```
# dmesg | grep HP
usb 5-1: Manufacturer: HP
HP CISS Driver (v 3.6.20)
usb 5-2: Manufacturer: HP
```



```
device with106.0 entered promiscuous mode
ehci5: port 3(with106.0) entering forwarding state
...done
Starting CT 107: CT: 107: started
device with107.0 entered promiscuous mode
ehci5: port 4(with107.0) entering forwarding state
...done
Starting CT 108: CT: 108: started
device with108.0 entered promiscuous mode
ehci5: port 5(with108.0) entering forwarding state
...done
Starting CT 181: CT: 181: started
device with181.0 entered promiscuous mode
ehci5: port 6(with181.0) entering forwarding state
...done
Starting CT 183: CT: 183: started
device with183.0 entered promiscuous mode
ehci5: port 7(with183.0) entering forwarding state
...done
Starting CT 184: CT: 184: started
device with184.0 entered promiscuous mode
ehci5: port 8(with184.0) entering forwarding state
...done
Starting CT 180: CT: 180: started
```

Figure 9: Boot messages

```
usb 5-2.2: Manufacturer: HP
input: HP Virtual Keyboard as /devices/pci0000:00/0000:00:
1e.0/0000:01:04.4/usb5/5-1/5-1.1.0/input/input1
generic-usb 0003:03F0:1027.0001: input,hidraw0: USB HID v.01
Keyboard [HP Virtual Keyboard] on usb-0000:01:04.4-1/input0
input: HP Virtual Keyboard as /devices/pci0000:00/0000:00:
1e.0/0000:01:04.4/usb5/5-1/5-1.1.1/input/input2
generic-usb 0003:03F0:1027.0002: input,hidraw1: USB HID v.01
Mouse [HP Virtual Keyboard] on usb-0000:01:04.4-1/input1
scsi 2:0:0:0: CD-ROM HP Virtual DVD-ROM PQ: 0 ANSI: 0 CCS
```

In the above code-snippet, the HP RAID driver is registered as 'CISS'. Let's repeat the *grep*, but with 'ciSS' as the filter:

```
# dmesg | grep ciSS
cciss 0000:25:00.0: PCI INT A-> GSI 34 (level, low) -> IRQ 34
cciss 0000:25:00.0: irq 69 for MSI/MSI-X
cciss 0000:25:00.0: irq 70 for MSI/MSI-X
cciss 0000:25:00.0: irq 71 for MSI/MSI-X
cciss 0000:25:00.0: irq 72 for MSI/MSI-X
IRQ 71/cciss0: IRQF_DISABLED is not guaranteed on shared IRQs
cciss0: <0x3230> at PCI 0000:25:00.0 IRQ 71 using DAC
cciss/c0d0: unknown partition table
cciss 0000:02:00.0: PCI INT A-> GSI 16 (level, low) -> IRQ 16
cciss 0000:02:00.0: irq 73 for MSI/MSI-X
cciss 0000:02:00.0: irq 74 for MSI/MSI-X
cciss 0000:02:00.0: irq 75 for MSI/MSI-X
cciss 0000:02:00.0: irq 76 for MSI/MSI-X
IRQ 75/cciss1: IRQF_DISABLED is not guaranteed on shared IRQs
cciss1: <0x3230> at PCI 0000:02:00.0 IRQ 75 using DAC
cciss/c1d0: pl pZ
cciss/c1d1: unknown partition table
```

As you see, this gives us two partitions found on one RAID controller (*cciss/c1d0*), and an empty partition table for the other. There's obviously a destroyed MBR there. We can try to mount the first partition now: